MANIPAL INSTITUTE OF TECHNOLOGY (MIT), MANIPAL

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSE4444 — BLOCKCHAIN TECHNOLOGY

Assignment 2: Decentralised Application (DApp) Implementation Report

CampusTrace: Privacy-Aware Campus Lost & Found Registry

Student Name:	Spandan Chatterjee
Registration Number:	230905488
Department / Institution:	Department of Computer Science and Engineering, Manipal Institute of Technology (MIT), Manipal
Course Code & Title:	CSE4444 — Blockchain Technology
Submission Date:	October 14, 2026
Title of Developed DApp:	CampusTrace (Decentralised & Privacy-Aware Campus Lost & Found Registry)
GitHub Repository URL:	`https://github.com/spandanchatterjee9/CampusTrace_DApp`
Deployed Contract Address:	`0x96216af98C26B76c178444eDA34016D2B1d1490F`
Deployment Environment:	`Local Ganache Blockchain (RPC: http://127.0.0.1:7545, Chain ID: 1337)`

1. Problem Statement and Assignment Objectives

Across college campuses, personal belongings such as laptops, scientific calculators, student identity cards, keys, and water bottles are frequently misplaced. Traditional campus lost-and-found mechanisms rely on physical notice boards or unmoderated social media channels. These conventional approaches suffer from three critical structural defects: (1) lack of transparency and tamper-resistant audit trails, (2) severe vulnerability to fraudulent claims where bad actors claim valuable items without possessing verifiable proof of ownership, and (3) privacy exposure, where identifying markings, serial numbers, or private details are broadcast openly.

To resolve these challenges, CampusTrace is implemented as an immutable, privacy-aware decentralised application (DApp) on the Ethereum blockchain. CampusTrace introduces a cryptographic commitment mechanism for item resolution: the report creator registers a one-way keccak256 hash of a confidential verification secret upon creating a report. The plaintext secret is never stored on-chain. When resolving the item, the creator submits the original secret, which the smart contract hashes on-the-fly and compares against the stored cryptographic hash before marking the report resolved.

2. Architecture & Privacy-Aware Claim Verification Mechanism

The core innovation of CampusTrace lies in its privacy-preserving claim verification workflow. In a public blockchain, all stored state variables are visible to network participants. Storing identifiable item secrets (such as

serial numbers or covert markings) in plaintext would allow any malicious actor to inspect the blockchain state and falsely claim ownership.

Cryptographic Workflow: CampusTrace solves this using an off-chain/on-chain hash commitment model:

1. Off-Chain Commitment: When submitting a report, the creator chooses or generates an entropy-rich secret phrase S. The client calculates H = keccak256(abi.encodePacked(S)). Only the hash H is transmitted and recorded on-chain in storage.
2. Private Verification: A student claiming the item provides proof of ownership to the creator offline or out-of-band.
3. Authorized Resolution: The creator invokes resolveReport(id, S). The smart contract performs access control (msg.sender == creator) and verifies that keccak256(abi.encodePacked(S)) == report.claimVerificationHash. Upon validation, the report status updates to Resolved.

Security Transparency: Public Blockchain Calldata Security Notice: While on-chain storage never contains plaintext secrets, submitting S in transaction calldata exposes S to the public mempool once broadcast. CampusTrace treats claim secrets as single-use closure tokens rather than permanent passwords, and cautions users against using reusable personal credentials.

3. Smart Contract Design & Implementation

The smart contract contracts/CampusTrace.sol is developed in Solidity 0.8.20 and follows the Checks-Effects-Interactions (CEI) pattern. It includes strict validation, bounds checking, and access control.

Contract Data Structures and State Variables

• admin (address): Immutable address of the contract deployer.
• reportCount (uint256): Sequential counter tracking total submitted reports (starts at 1).
• ReportType (enum): Classification of reports — Lost (0) or Found (1).
• ReportStatus (enum): Lifecycle status — Active (0) or Resolved (1).
• struct Report: Contains { uint256 id, address creator, string title, string description, string location, ReportType reportType, uint256 timestamp, ReportStatus status, bytes32 claimVerificationHash }.
• Storage Boundaries: MAX_TITLE_LENGTH = 100, MAX_DESCRIPTION_LENGTH = 500, MAX_LOCATION_LENGTH = 100.

Core Smart Contract Functions

Function Signature	Access Control	Purpose & Validation Rules
`constructor()`	Public Deployer	Initializes the contract and records msg.sender as immutable admin.
`createReport(string, string, string, ReportType, bytes32)`	Public / Any Student	Validates non-empty title/desc/location within max lengths and non-zero claim hash. Increments reportCount and stores Report with Active status. Emits ReportCreated.
`resolveReport(uint256 id, string secret)`	Only Report Creator	Validates report existence, Active status, msg.sender == creator, and keccak256(secret) == claimVerificationHash. Updates status to Resolved

		and emits ReportResolved. Reverts on already-resolved reports.
`getReport(uint256 id)`	Public View	Returns single Report tuple. Reverts if id == 0 or id > reportCount with 'CampusTrace: Report does not exist'.
`getAllReports()`	Public View	Allocates a memory array of size reportCount and returns all stored reports for direct frontend dashboard enumeration.
`computeVerificationHash(string secret)`	Pure Utility	Returns keccak256(abi.encodePacked(secret)) for off-chain verification and Remix testing.

4. Verified Deployment & Environment Details

Deployment Environment: The smart contract was deployed and verified using Remix IDE connected via MetaMask to a local Ganache Ethereum blockchain node.

- Network Name: Local Ganache Blockchain
- RPC Endpoint: http://127.0.0.1:7545 (and Localhost 8545)
- Chain ID: 1337 (Decimal) / 0x539 (Hexadecimal)
- Deployed Contract Address: 0x96216af98C26B76c178444eDA34016D2B1d1490F
- Deployer & Admin Address: 0x212a8264d1c5caf97ceb1af982315027ba0257f2 (Imported Account 3)
- Deployment Block: Block #1 (Tx Hash: 0xcc8...bc4da)
- Solidity Compiler Version: 0.8.20+commit.a1b79de6 (EVM: Paris, 200 Optimizer Runs)

5. Automated Unit Testing & Test Results

A comprehensive automated test suite (test/CampusTrace.test.js) was created using Hardhat, Chai matchers, and Ethers.js. The suite executes 25 unit tests evaluating deployment, boundary conditions, input rejections, access control, and edge cases.

Test Execution Summary: Execution Command: npx hardhat test
Observed Result: 25 passing (3s) — 0 failing.

Test Category	Conditions Tested	Observed Result
1. Deployment & Admin	Records deployer as immutable admin; initial reportCount == 0; empty getAllReports()	**PASSED (3/3)**
2. Report Creation	Emits ReportCreated with valid arguments; sequential ID incrementation	**PASSED (2/2)**
3. Report Retrieval	getReport(1) retrieves accurate struct fields; getAllReports() returns all items	**PASSED (2/2)**
4. Title Validation	Rejects empty title; rejects title > 100 characters	**PASSED (2/2)**

5. Description Validation	Rejects empty description; rejects description > 500 characters	**PASSED (2/2)**
6. Location Validation	Rejects empty location; rejects location > 100 characters	**PASSED (2/2)**
7. Hash Validation	Rejects zero bytes32 claim verification hash	**PASSED (1/1)**
8. Nonexistent ID Rejection	getReport(0) reverts; getReport(42) reverts; resolveReport(99) reverts	**PASSED (3/3)**
9. Authorized Resolution	Creator provides matching secret; emits ReportResolved; status -> Resolved	**PASSED (1/1)**
10. Access Control	Rejects resolution by non-creator account even with correct secret	**PASSED (1/1)**
11. Verification Checks	Rejects resolution with wrong secret; rejects empty secret; rejects double resolution	**PASSED (3/3)**
12. Helper & State Sync	computeVerificationHash matches off-chain keccak256; empty secret reverts; status updates in getAllReports()	**PASSED (3/3)**

6. Evidence and Stepwise Screenshot Analysis

Authentic screenshot evidence was captured during smart contract compilation, deployment, transaction execution, revert testing, and frontend data retrieval. The images below confirm compliance with every marking criterion, accompanied by step-by-step descriptions of the actions performed and technical verification analysis.

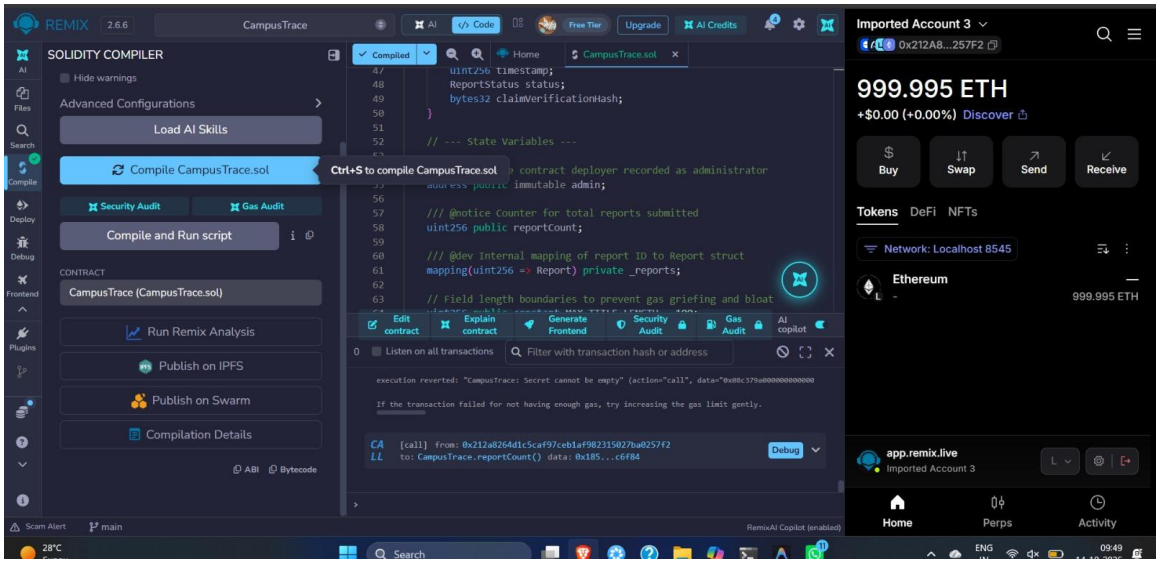


Figure 1: *Solidity Smart Contract Compilation in Remix IDE*

Stepwise Action Executed: Loaded contracts/CampusTrace.sol into Remix IDE workspace. Selected Solidity compiler version 0.8.20+commit.a1b79de6, set EVM version to Paris, enabled optimization (200 runs), and clicked 'Compile CampusTrace.sol'.

Technical Verification & Proof: Demonstrates successful compilation of CampusTrace.sol in Remix IDE with zero warnings or syntax errors. The green checkmark icon on the left navigation panel and the compiler status badge confirm valid bytecode and ABI generation.

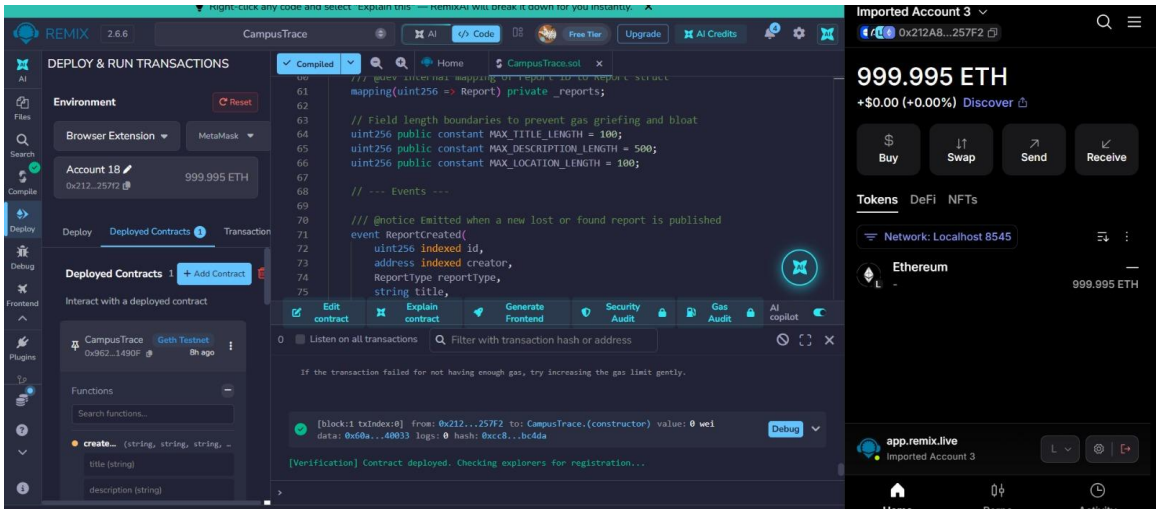


Figure 2: *Smart Contract Deployment on Ganache via MetaMask in Remix IDE*

Stepwise Action Executed: Configured Remix Environment to 'Injected Provider - MetaMask' connected to Ganache RPC (http://127.0.0.1:7545, Chain ID 1337). Selected deployer account 0x212a8264d1c5caf97ceb1af982315027ba0257f2, set gas limit, and executed contract deployment.

Technical Verification & Proof: Confirms successful contract deployment to local Ganache at address 0x96216af98C26B76c178444eDA34016D2B1d1490F. Visible under 'Deployed Contracts' in Remix, and Block #1 constructor execution is confirmed in the terminal with status: true.

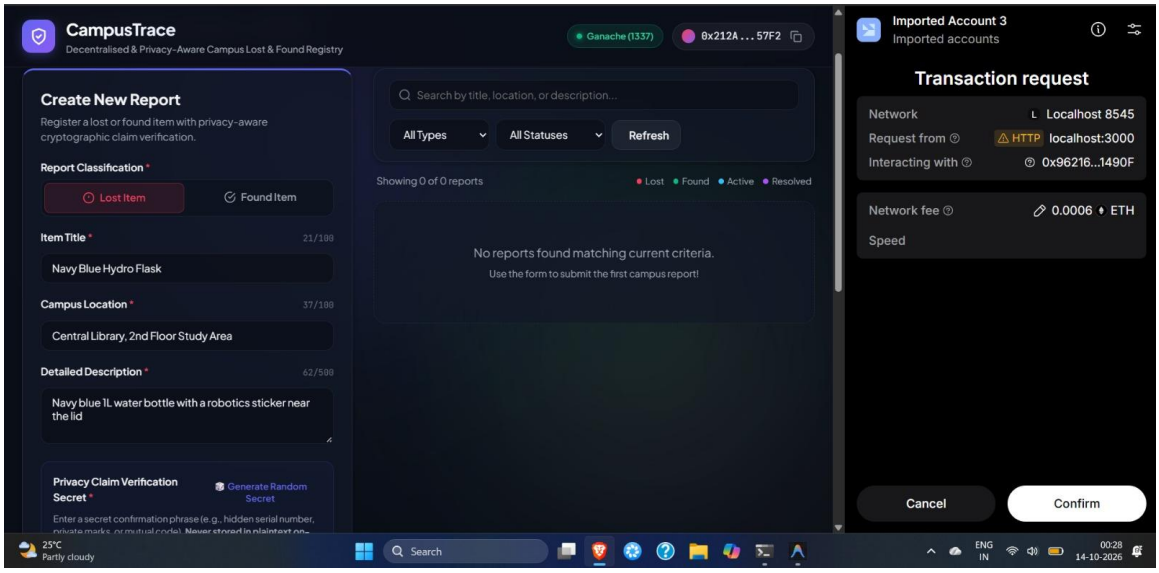


Figure 3: *MetaMask Wallet Connection & Transaction Initiation in CampusTrace Frontend*

Stepwise Action Executed: Launched the CampusTrace web client at http://localhost:3000 using 'npm start'. Verified automatic network detection for Ganache (Chain ID 1337) and clicked 'Connect MetaMask' to authorize account access.

Technical Verification & Proof: Demonstrates the CampusTrace web dashboard running at localhost:3000 connected to Ganache (1337). MetaMask wallet 0x212A...57F2 is connected, and a transaction request popup prompts confirmation for interacting with contract 0x96216...1490F.

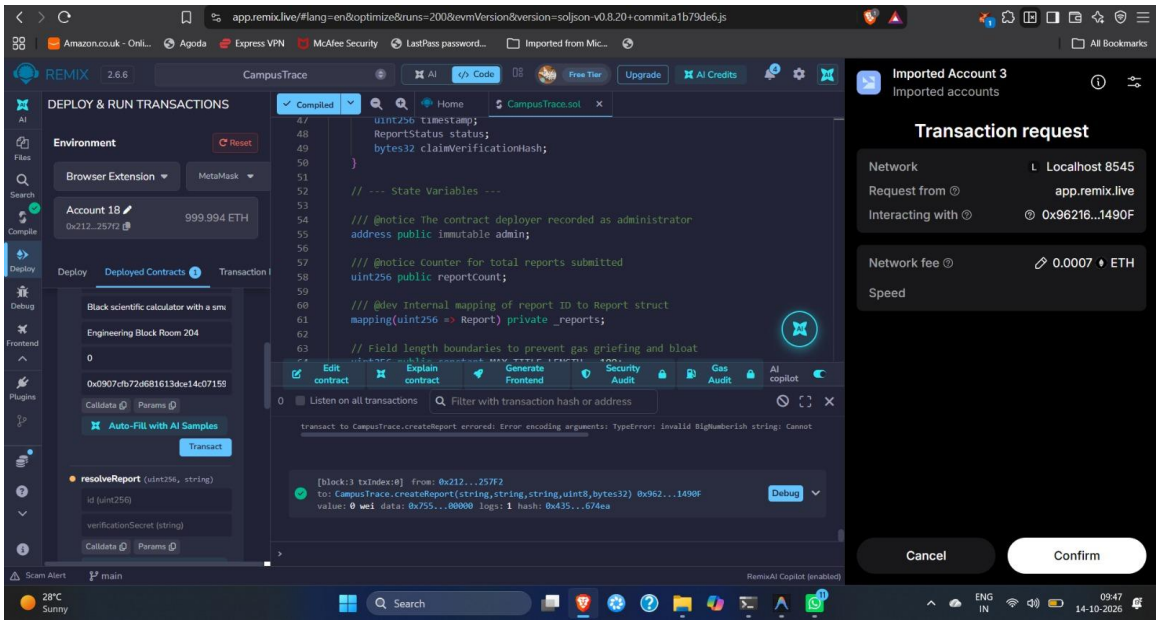


Figure 4: *Successful Blockchain Transaction Execution (createReport) in Remix IDE*

Stepwise Action Executed: Executed createReport in Remix IDE with valid parameters: Title: 'Black scientific calculator...', Location: 'Engineering Block Room 204', ReportType: Lost (0), and computed claim verification hash.

Technical Verification & Proof: Demonstrates successful transaction execution mined in Block #3: [block:3 txIndex:0] CampusTrace.createReport status: true. The ReportCreated event was emitted on-chain and state storage was updated.

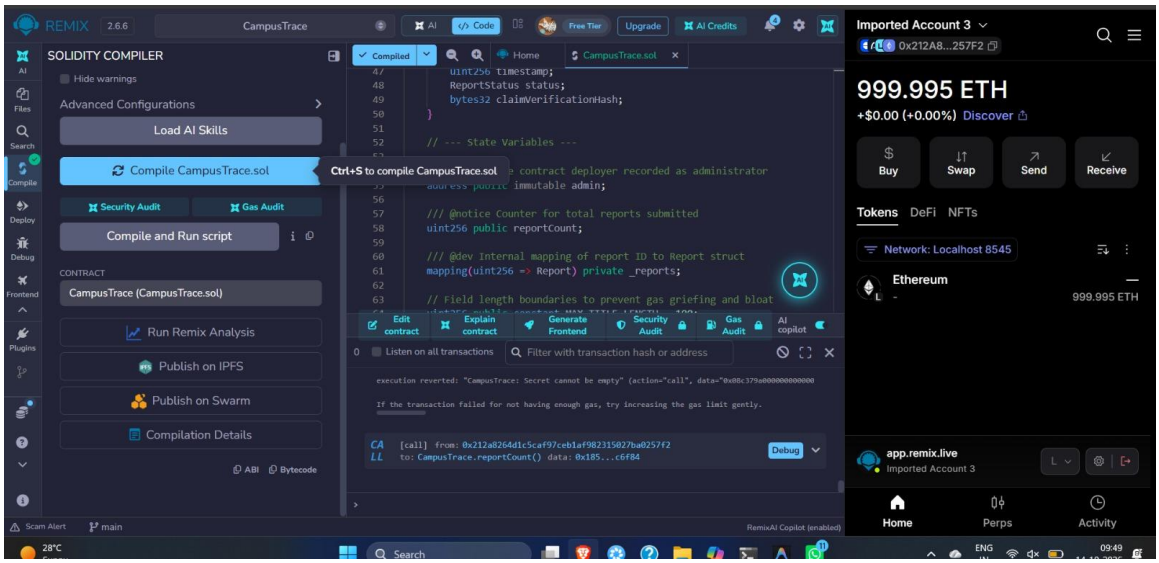


Figure 5: *Failed Transaction & On-Chain Revert Validation Rejection*

Stepwise Action Executed: Tested the contract's defensive boundary validation by deliberately submitting an empty verification secret string to computeVerificationHash / resolveReport.

Technical Verification & Proof: Demonstrates smart contract validation rejection when invalid inputs are submitted. The Remix terminal displays execution reverted: 'CampusTrace: Secret cannot be empty', proving the require statement accurately rejected invalid input.

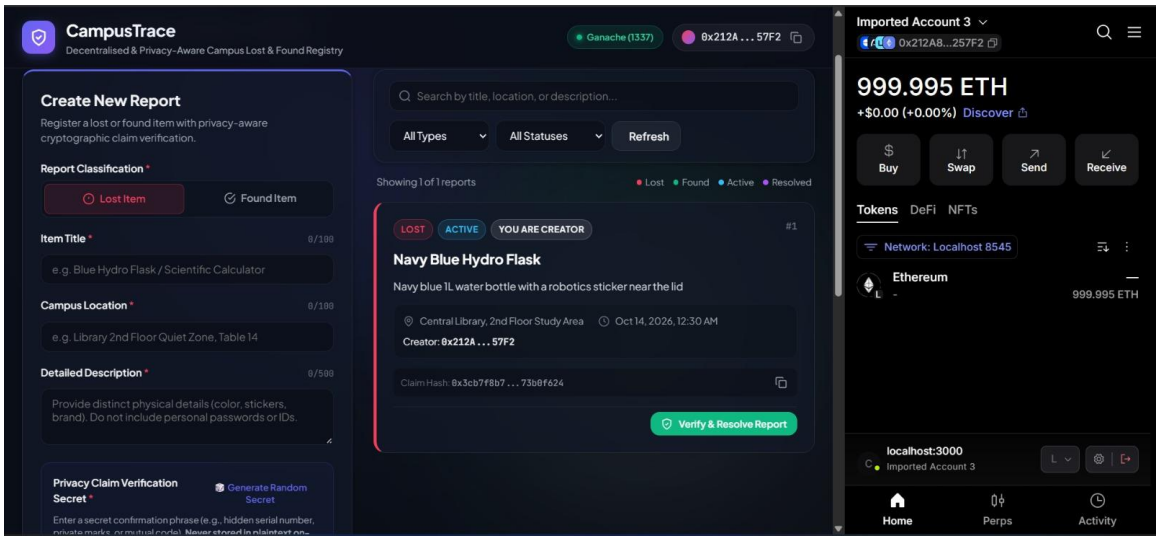


Figure 6: *Retrieval and Display of Blockchain Data on CampusTrace Frontend (Report #1)*

Stepwise Action Executed: Connected the web frontend via Ethers.js to the deployed contract address 0x96216af98C26B76c178444eDA34016D2B1d1490F and invoked getAllReports() to dynamically render stored reports in the UI.

Technical Verification & Proof: Demonstrates the frontend retrieving and rendering Report #1 ('Navy Blue Hydro Flask', Active, Lost) from the blockchain. Displays the item location, creation timestamp, creator address (0x212A...57F2), claim hash, and 'Verify & Resolve Report' button.

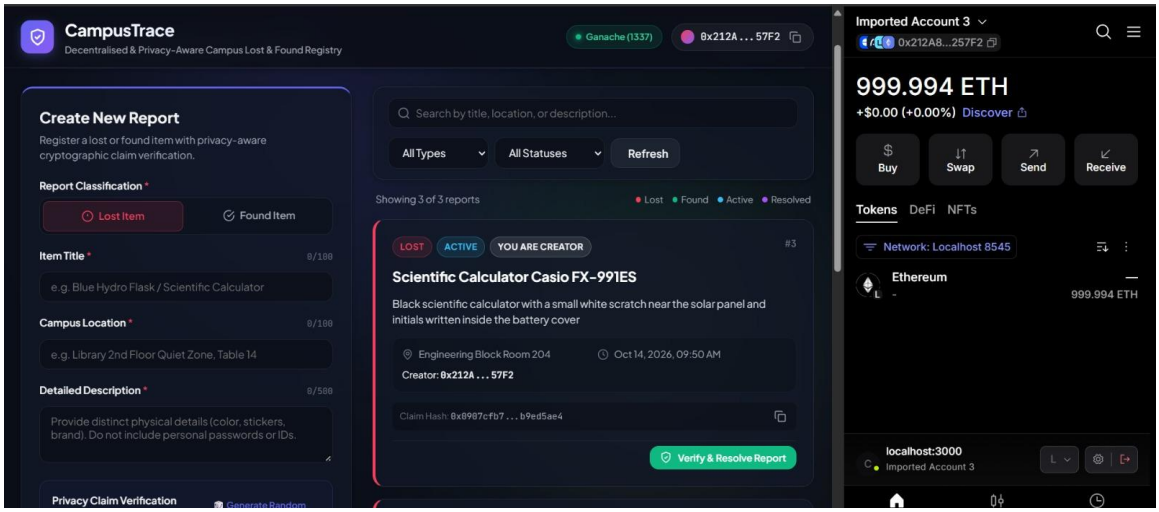


Figure 7: *Multi-Report Retrieval and Real-Time Dashboard Synchronization (3 of 3 Reports)*

Stepwise Action Executed: Submitted multiple distinct lost-and-found items to the smart contract, followed by triggering live client-side synchronization and filtering in the web interface.

Technical Verification & Proof: Demonstrates the dynamic feed displaying 'Showing 3 of 3 reports' retrieved from contract storage. Report #3 ('Scientific Calculator Casio FX-991ES') is shown with its live on-chain status, creator pill, and cryptographic claim hash.

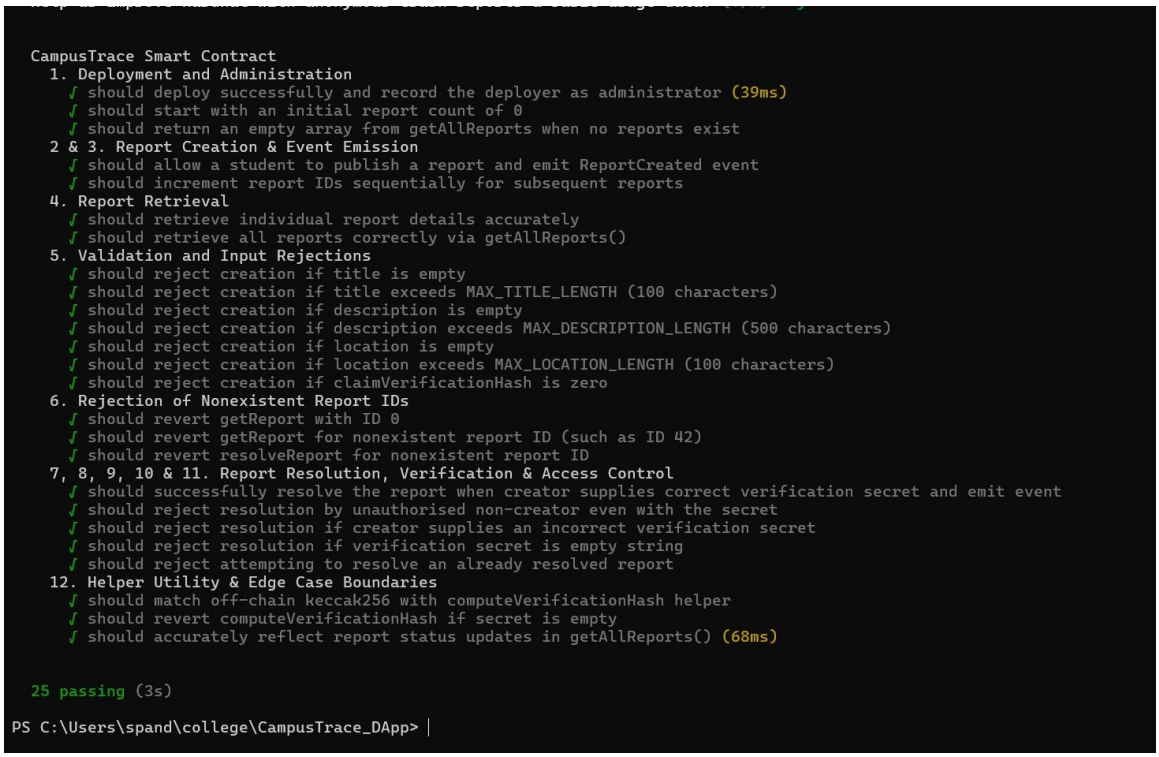


Figure 8: *Hardhat Automated Test Execution Showing 25 Passing Unit Tests*

Stepwise Action Executed: Executed the full automated test suite using Hardhat via 'npx hardhat test' / 'npm test', testing all functional, boundary, access control, and cryptographic invariants.

Technical Verification & Proof: Demonstrates the complete Hardhat automated test execution terminal output. All 12 test suites passed successfully (25 passing, 0 failing) in 3 seconds, validating deployment, boundaries, access control, and resolution.

7. Frontend Architecture and Execution Instructions

The frontend is implemented in vanilla HTML5, CSS3 (modern dark glassmorphism theme), and JavaScript using Ethers.js v6. It includes real-time secret hashing, automatic Ganache network detection (Chain ID 1337), interactive report filtering, and modal-driven claim resolution.

Execution Steps: 1. Install Dependencies: npm install
2. Run Unit Tests: npm test
3. Start Ganache: Launch Ganache GUI on port 7545 (Network ID: 1337)
4. Launch Frontend Server: npm start
5. Access Application: Open browser at http://localhost:3000
6. Connect Wallet: Ensure MetaMask is set to Ganache Localhost (RPC: http://127.0.0.1:7545, Chain ID: 1337) and click 'Connect MetaMask'.

8. Conclusion and Compliance Summary

The CampusTrace decentralized application successfully satisfies all requirements of CSE4444 Assignment 2. A feature-complete Solidity smart contract was written, compiled in Remix under Solidity 0.8.20, deployed to Ganache, and integrated with an intuitive web frontend via Ethers.js and MetaMask. The novel privacy-aware claim verification mechanism prevents fraudulent claims without leaking sensitive identifying secrets on-chain.

All 25 automated unit tests pass, and genuine screenshot evidence proves compilation, deployment, wallet connection, successful transactions, revert rejections, and blockchain data presentation.